

Developing a Reproducible, Interactive Statistics Course

Jupyter notebooks, Git, Colab, VS Code, Overleaf, HPC, and AI-supported course maintenance

Daan Meerburg

University of Groningen

Academic Staff Presentation

What I Built

- New statistics course for third-year BSc Physics students
- 12 Jupyter lecture notebooks
- Colab to execute in the cloud
- Alt: VS Code/Git route for durable local editing
- Board notes, quizzes, assignments, projects, exams, rubrics
- Development workflow on the HPC, synchronized with Git and Overleaf

Central format

Jupyter notebooks

Notebook contents

text, equations, code, figures, exercises

Student access

Colab or local VS Code/Git

Keep these together

- Theory and Math
- Python implementation
- Plots and numerical output
- Execution and interpretation

Students can move between formulas, code, and interpretation in the same environment.

Why notebooks

A Jupyter notebook puts explanation, code cells, output, and short checks in one document.

In class

Students can rerun an example while the statistical idea is being discussed.

Design Principle: One Executable Document

Each lecture notebook combines:

- Text and equations
- Python code
- Figures and simulations
- Short exercises and larger assignments
- Links to data and external resources

The lecture note is also the file students run and edit.

Notebook as lecture note

explanation, notation, examples

Notebook as executable file

runnable code, plots, data

Notebook as exercise space

change assumptions and rerun

Two Student Routes

Route 1: Colab

- click the Colab badge
- run cells immediately
- many common Python libraries already available
- use Drive/download/GitHub export for lasting copies

Route 2: VS Code + Git

- Fork or clone the repository
- Edit notebooks locally
- Commit and maintain a personal version
- More package setup, more flexibility

Trade-off

Colab is easiest for class use, but runtime state is temporary. VS Code and Git require local setup, but give students a persistent fork they can edit and maintain.

Lectures 1–4

- Introduction
- Probability
- Distributions
- Correlations

Lectures 5–7

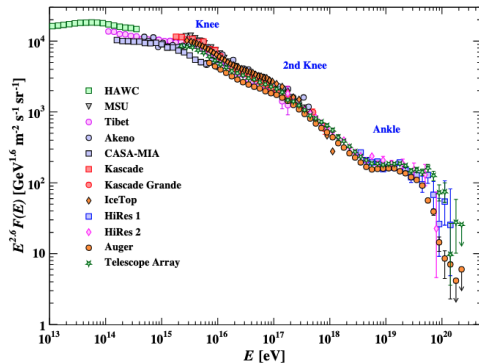
- Likelihood functions
- Maximum likelihood estimation (MLE)
- Bootstrap and jackknife
- Outliers and model comparison

Lectures 8–12

- Priors and posteriors
- Maximum a posteriori (MAP) estimation
- Marginalization
- Gibbs sampling
- MCMC and nested sampling

foundations → frequentist inference → Bayesian inference

Example Visuals from the Course



Teaching use

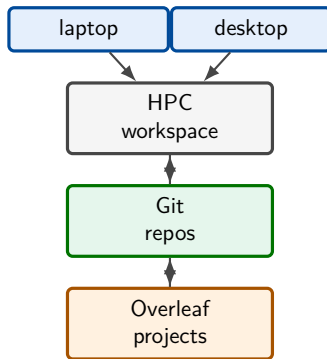
- Students run the same examples
- Plots are tied to code
- Assumptions can be changed
- Interpretation follows directly
- Student can run and modify the notebook in their own time

Data, uncertainty, and scaling laws appear directly in the notebook.

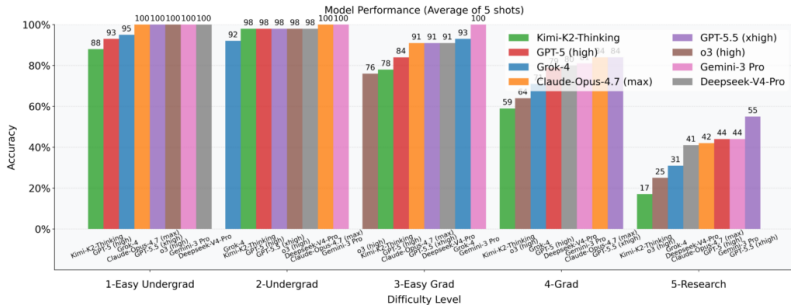
Why use HPC for Development?

My development workspace is centered on the HPC:

- Accessible from all machines
- Course repository cloned and version controlled
- Overleaf projects linked through Git
- Notebooks, notes, assignments, exams, and rubrics in one workflow
- Easy to maintain and update the course for future years



Why Try AI in This Course?



TPBench

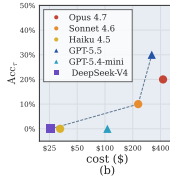
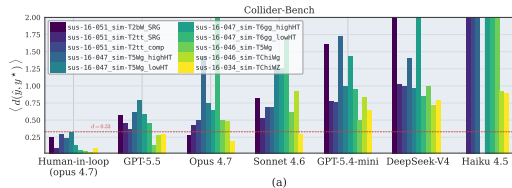
Theoretical-physics problems from undergraduate to research level.

Takeaway

GPT-5.5 is among the strongest models, with performance even on research-level problems. BSc problems are almost 100% solved.

Source: TPBench, May 2026.

Scientific Coding Agents Are Being Benchmarked



Collider-Bench

Agents reproduce LHC analyses using public papers and open software.

Takeaway

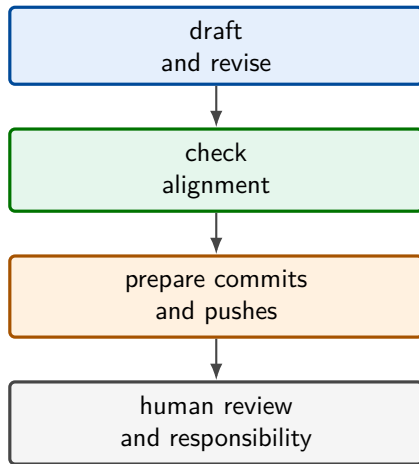
GPT-5.5 is competitive, but the paper still finds a gap to physicist-in-the-loop workflows.

Source: Faroughy et al., arXiv:2605.13950, Fig. 2.

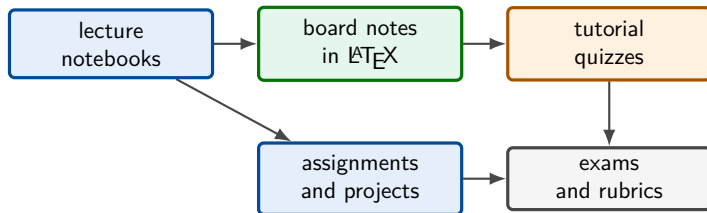
Where AI Fit

I used AI for specific course-development tasks:

- Revise notebook explanations
- Turn notebook material into board-style notes
- Draft assignments, projects, exams, and rubrics
- Check consistency across course components
- Maintain repositories: diffs, commits, pushes, public/private separation
- List is growing as new AI tools become available and improve



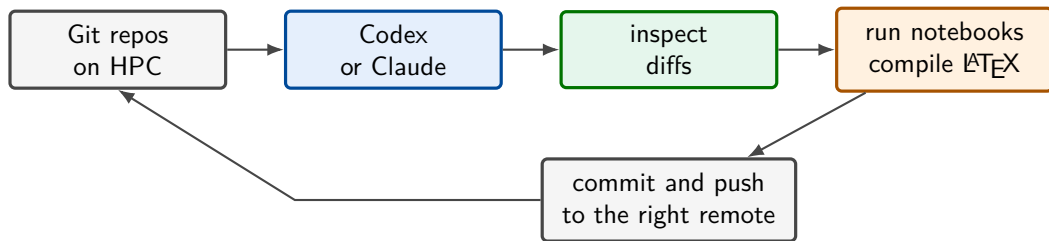
From Lecture to Assessment



Design goal

Keep explanation, practice, project work, and assessment aligned.

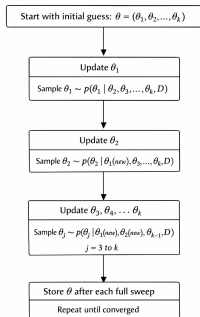
AI-Supported Repository Workflow



Control point

The useful part is not automation by itself. The useful part is that changes are visible as diffs, notebooks can be rerun, L^AT_EX can be compiled, and I decide what is released.

Inference Examples in Notebooks



Gibbs sampling

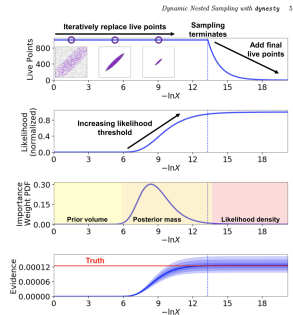


Figure 2: An example illustrating the behavior of a Markov Monte Carlo sampling run in *sparsity*. Top: the evolution of the points in a hierarchy of prior $\mathcal{E}_k$ ’s. Squiggles of color indicate their classification [argmax] with respect to the current best hypothesis. Top-left: $\mathcal{E}_0$ ’s are highlighted in several lines. The number of five points remains constant until sampling terminates, at which point we add the first five points one-by-one to the samples. Top-middle: The (unnormalized) likelihood $L(\mathbf{z})$ associated with a prior value $\mathcal{E}_k(\mathbf{z})$ in the top panel. This increase monotonically as we sample increasingly smaller regions of the prior. Bottom-middle: The importance weight PDF $\mathcal{P}(\mathbf{z})$, roughly divided into regions dominated by the prior value $\mathcal{E}_k$ is large, $L(\mathbf{z})$ is small; $L(\mathbf{z})$ is small; $\mathcal{E}_k$ is small; posterior mean $\hat{\mathbf{z}}(\mathbf{z})$ and $L(\mathbf{z})$ are comparable; orange; and likelihood dominated $\hat{\mathbf{z}}(\mathbf{z})$ is small, $L(\mathbf{z})$ is large; red. The posterior mean is the most likely value of $\mathbf{z}$ given the data and prior. Bottom-right: The estimated evidence $\mathcal{Z}_k$ for each hypothesis. Bottom: The estimated evidence $\mathcal{Z}_k$ for each hypothesis. For $i = 1, 2$, and $\mathbf{z}_{\text{true}}$ mean (blue squiggle). The true value is shown in red.

Nested sampling

Benefits for Teaching

- Students can rerun computational examples.
- Exercises become closer to real research practice.
- Lectures, assignments, projects, and exams stay aligned.
- Updates are easier because the course is version controlled.
- Colleagues can inspect, reuse, or adapt parts of the course.

reproducibility

editable copies

maintainability

research practice

Things to keep in mind

Challenges

- Colab sessions are convenient but temporary.
- VS Code/Git gives flexibility, but needs setup.
- Notebooks can hide complexity.
- AI can produce plausible errors.

Mitigations

- Explicit persistence/forking instructions (lecture 1)
- During lecture I combine blackboard presentation with live notebook execution
- Students are encouraged to read and understand the code, not just run it
- AI is used as a drafting tool, but all changes are reviewed before release
- Private repos for exams and solutions, public repos for lectures and assignments

The course is built around version-controlled Jupyter notebooks. It uses many modern tools to make the material interactive, reproducible, and maintainable. AI is a helpful assistant, but human review and control are essential.

- Students can read, run, fork, and modify the lectures.
- They can choose Colab for access or VS Code/Git for persistence.
- Git, Overleaf, and the HPC make the material easier to maintain.
- AI helps with drafting and repository maintenance, but changes are reviewed before release.